

SLURM

Simple Linux Utility for Resource Management

From First Principles to Production-Scale AI/ML Deployment

Internal Infrastructure Documentation

Distributed Systems & HPC Infrastructure Team

Version 1.0 | Confidential

Section 1 — Foundations of Cluster Scheduling

1.1 Why Cluster Schedulers Exist

Modern scientific and machine-learning workloads require far more compute than any single machine can provide. A GPU training run for a large language model may require hundreds of A100 or H100 accelerators running in tight synchrony for days or weeks. Without a cluster scheduler, coordinating this kind of work is manual, error-prone, and inefficient: engineers would need to negotiate access to nodes, hand-configure processes, and monitor runs themselves.

Cluster schedulers solve the fundamental resource-sharing problem: multiple users and workloads compete for a finite pool of heterogeneous hardware (CPUs, GPUs, memory, network bandwidth). A scheduler acts as the arbiter, accepting job submissions, deciding when and where each job runs, enforcing policies, and reporting outcomes.

The historical origin of HPC schedulers lies in batch processing systems developed for scientific supercomputers in the 1980s and 1990s. Early systems like PBS (Portable Batch System) and LSF (Load Sharing Facility) established the batch-queue paradigm that SLURM inherits and extends. SLURM was originally developed at Lawrence Livermore National Laboratory beginning around 2002, specifically to address the needs of massively parallel scientific computing.

1.2 HPC Cluster Architecture Overview

A typical HPC cluster consists of several categories of nodes, each with a distinct role:

- **Login Nodes (Head Nodes):** User-facing entry points where jobs are submitted. Users SSH into these nodes to prepare data, write scripts, and invoke `squeue/sbatch`. Login nodes are never used for computation.
- **Compute Nodes:** The primary workers. Each compute node has CPUs, RAM, and optionally GPUs or other accelerators. SLURM places jobs on compute nodes according to scheduling policy.
- **Controller Node:** Runs the `slurmctld` daemon, which maintains the global cluster state, schedules jobs, and manages the job queue. High-availability setups run a standby controller.
- **Database Node:** Runs `slurmdbd`, the SLURM database daemon, which persists accounting records, job history, and user/group associations to a MySQL/MariaDB backend.
- **Storage Nodes / Parallel File System:** Shared filesystem (Lustre, GPFS, BeeGFS) accessed by all nodes. Provides the working directory for job inputs and outputs.
- **Management / BMC Network:** Out-of-band network (IPMI/BMC) for power management, console access, and hardware monitoring independent of the primary data network.

Node Type	Key Role	Typical Hardware
Login Node	Job submission, user shell access	2–4 CPU cores, 32–64 GB RAM
Compute Node (CPU)	Batch compute workloads	2× 64-core CPUs, 256–512 GB RAM

Node Type	Key Role	Typical Hardware
Compute Node (GPU)	Deep learning training/inference	8× A100/H100, NVLink, 80 GB HBM2e each
Controller	Cluster scheduling & state	2× 32-core CPUs, 256 GB RAM (HA pair)
Database	Accounting persistence	16-core CPU, 128 GB RAM, SSD RAID
Storage	Shared parallel file system	Lustre OSS, 2+ PB usable

1.3 Problems Solved by Schedulers

Resource Allocation

Jobs declare their resource requirements (CPUs, GPUs, memory, time limit). The scheduler finds nodes that satisfy these requirements, assigns exclusive or shared access, and launches the job. Without a scheduler, two jobs would compete for the same GPUs causing failures or silent data corruption.

Fairness & Priority

In a multi-user cluster, fairness policies ensure no single user or group monopolises resources. SLURM implements fair-share scheduling: each account is allocated a share of the cluster, and heavy historical consumers are deprioritized relative to accounts that have used little. Priority is a composite score combining fair-share, job age, requested resources, and QOS (Quality of Service) factors.

Queueing & Backfill

Jobs waiting for resources are held in a queue (PENDING state). Simple FIFO queues starve small jobs behind large reservation-blockers. SLURM uses backfill scheduling: small jobs can jump ahead of large pending jobs if they can start and finish before the large job's reserved time slot begins, improving overall cluster utilization.

Gang Scheduling & Synchronous Launch

Distributed training jobs need all their processes (ranks) to start simultaneously. SLURM supports gang scheduling and step barriers, ensuring all compute nodes for a job are allocated before any task launches, which is critical for MPI and PyTorch Distributed applications.

Preemption

High-priority jobs (e.g., from a QOS with preempt rights) can evict lower-priority running jobs. SLURM supports multiple preemption mechanisms: checkpoint-and-requeue, suspend, and cancel. This enables burst-capacity scenarios and priority classes.

1.4 Comparison with Kubernetes Scheduling

Dimension	SLURM	Kubernetes
Primary paradigm	Batch HPC jobs, tightly coupled MPI	Long-running microservices & loosely coupled jobs
Scheduling unit	Job / Job Step (MPI ranks)	Pod (container group)
Resource model	Node-level: CPUs, GPUs, memory, GRES	Container-level requests/limits
Gang scheduling	Native, synchronous allocation	Via volcano/kueue plugins (complex)
Preemption	First-class, configurable per QOS	Priority classes + PriorityPreemption
Accounting	Built-in job accounting via slurmdbd	External (Prometheus/custom)
Networking	Bare-metal MPI/InfiniBand native	Overlay CNI (Flannel, Calico, SR-IOV)
Storage	Direct POSIX parallel FS (Lustre)	PVC/CSI, not optimized for parallel FS
Checkpointing	DMTCP, BLCR, CRIU integration	CRIU via Kubernetes annotations
GPU topology	NUMA & topology-aware GRES	Device plugin, limited topology
Overhead	Very low per-job overhead (<1s)	Higher pod scheduling latency

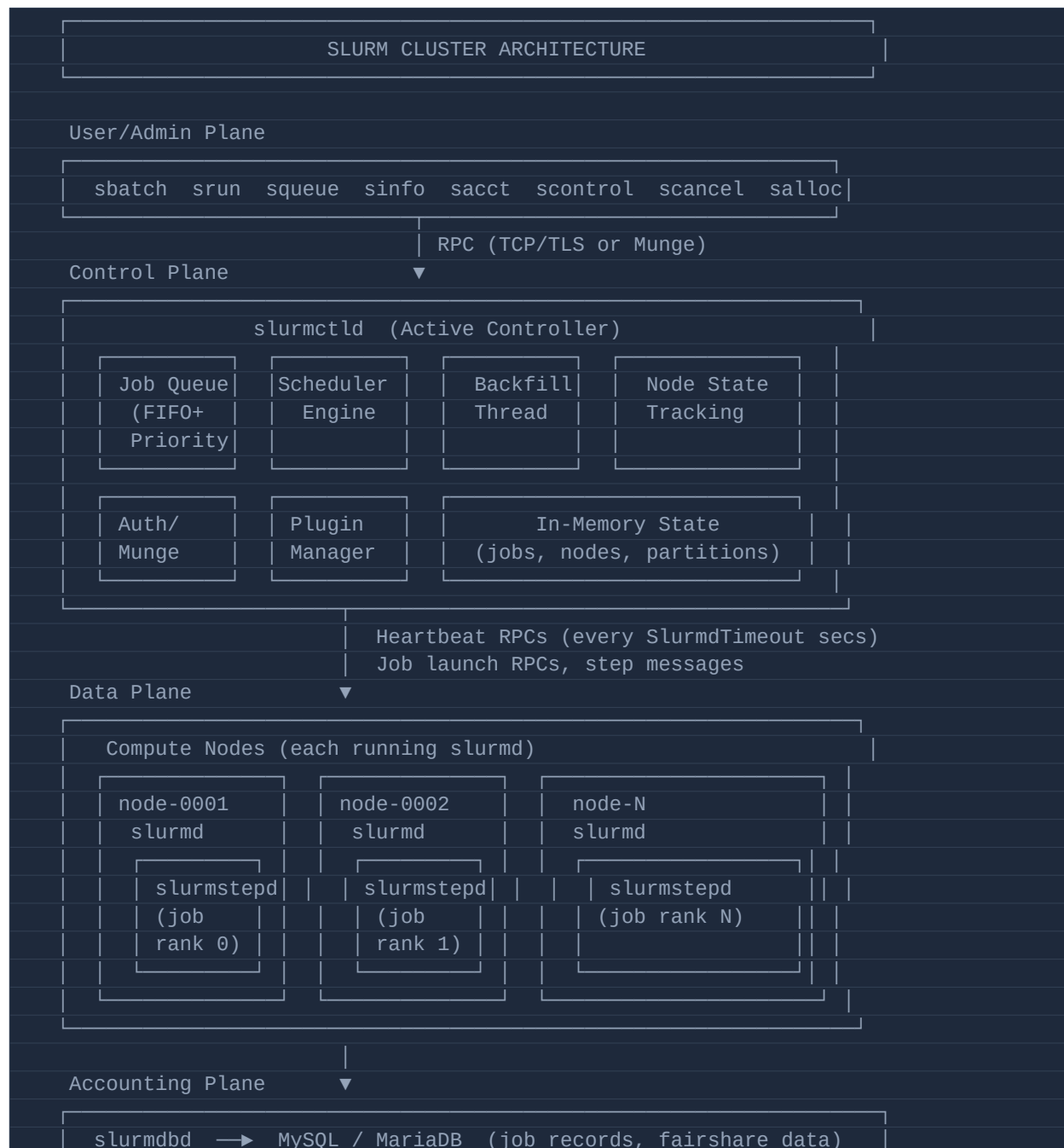
Key Insight

SLURM excels at tightly-coupled, resource-intensive batch jobs where performance isolation, InfiniBand fabric, and per-node GPU topology matter. Kubernetes is better suited for containerized microservices, elastic inference, and heterogeneous cloud-native pipelines. Production AI infrastructure often runs both: SLURM for training, Kubernetes for serving.

Section 2 — SLURM Architecture Deep Dive

2.1 Architectural Overview

SLURM follows a centralized controller architecture with distributed compute agents. The design separates concerns clearly: a single authoritative controller manages global state, while lightweight per-node daemons handle local execution. All communication flows through well-defined RPC channels.



2.2 slurmctld — The Central Controller

slurmctld is the brain of a SLURM cluster. It is a single multi-threaded daemon that runs on the controller node (or an HA pair). It maintains the authoritative state of all nodes, jobs, partitions, and reservations entirely in memory, checkpointing to disk periodically for recovery.

Internal Thread Architecture

- **Main thread:** Handles incoming RPCs from client commands (sbatch, srun, squeue) and from slurmd daemons.
- **Scheduler thread:** Runs the main scheduling loop at intervals defined by SchedulerParameters. Evaluates pending jobs against available nodes and launches ready jobs.
- **Backfill thread:** Runs periodically (default every 30 seconds) to perform look-ahead scheduling. Simulates future node availability to identify pending jobs that can be started without delaying higher-priority jobs.
- **Accounting thread:** Asynchronously sends job start/update/end records to slurmdbd to avoid blocking the critical scheduling path.
- **Node health thread:** Processes heartbeat messages from slurmd daemons and updates node states (IDLE, ALLOCATED, DOWN, DRAIN).
- **Checkpoint thread:** Periodically writes the full cluster state to disk so recovery after controller restart does not lose pending jobs.

2.3 slurmd — The Compute Node Daemon

slurmd runs on every compute node. It registers with slurmctld on startup, sends regular heartbeat RPCs, and receives job launch instructions. When a job is assigned to a node, slurmctld sends slurmd a launch message containing the job's resource allocation, environment, and executable.

slurmstepd — The Job Step Executor

When slurmd receives a job step launch request, it forks a child process: slurmstepd. This process takes over management of the job step, setting up cgroup isolation, binding processes to CPU/GPU affinity, setting environment variables, and exec'ing the user's job script. Using a separate slurmstepd process isolates job failures from slurmd: if a job crashes, slurmd continues serving other requests.

2.4 slurmdbd — The Database Daemon

slurmdbd provides a persistent accounting layer. It accepts records from one or more slurmctld instances (supporting multi-cluster federation), aggregates them, and stores them in a MySQL/MariaDB database. This enables:

- Historical job queries via sacct (CPU-hours consumed, efficiency, start/end times)
- Fair-share calculations based on past resource consumption

- Per-user and per-account usage reporting and limits enforcement
- Multi-cluster federation with global job visibility

2.5 Client Commands Reference

Command	Purpose	Key Flags
sbatch	Submit batch script to queue	--nodes, --gres, --partition, --time, --account
srun	Launch interactive or inline job step	--nodes, --ntasks, --gres=gpu:N, --pty
salloc	Allocate resources interactively	--nodes, --time, --partition
squeue	View job queue and status	--user, --partition, --format, --json
sinfo	View node and partition status	--partition, --format, --Node
scancel	Cancel running or pending jobs	--job, --user, --partition
sacct	Query accounting database	--jobs, --format, --user, --starttime
scontrol	Admin tool: inspect/modify state	show job, show node, update, suspend, resume
sdiag	Scheduler diagnostics & statistics	Backfill stats, scheduling cycle times

2.6 Communication Protocols & RPC Mechanisms

All SLURM daemon and client communication uses a custom binary RPC protocol over TCP. Messages are serialized using SLURM's internal pack/unpack library (not protobuf or JSON), which produces compact binary records. Every message is authenticated.

Authentication via Munge

SLURM uses Munge (a credential service) by default for authentication. Munge issues time-limited encrypted credentials signed with a shared secret (munge.key) distributed to all cluster nodes. Any daemon receiving an RPC verifies the Munge credential to confirm the sender's UID and cluster membership. This design requires a shared secret but avoids the complexity of PKI.

```
# Munge credential flow:
# 1. Client (e.g. sbatch) calls munge_encode() → receives credential blob
# 2. Credential embedded in every RPC header
# 3. Receiving daemon calls munge_decode() → validates UID, GID, timestamp
# 4. Credential rejected if >TTL (default 300s) or wrong cluster key

# Verify munge is working on all nodes:
munge -n | unmunge          # encode and decode on same node
munge -n | ssh nodeXXX unmunge # encode locally, decode remotely
```

Port Configuration

Port	Service	Protocol
6817 (default)	slurmctld (controller)	TCP, Munge-authenticated

Port	Service	Protocol
6818 (default)	slurmd (compute nodes)	TCP, Munge-authenticated
6819 (default)	slurmdbd (database daemon)	TCP, Munge-authenticated

2.7 State Management & Recovery

slurmctld is a stateful service. All cluster state (job queue, node states, reservations, partitions) is held in RAM for speed. Periodically (by default every 60 seconds), the controller writes a binary checkpoint file (state/job_state, state/node_state, etc.) to disk. On restart, slurmctld reads these files and rebuilds its in-memory state, resuming scheduling without losing pending or running jobs.

HA Consideration

In a single-controller setup, slurmctld downtime halts scheduling (running jobs continue, but no new jobs start). For production clusters, configure a standby controller using SlurmctldHost with two entries. The standby monitors the primary and takes over within seconds of a failure.

Section 3 — Job Lifecycle

3.1 Complete Job Lifecycle: Step-by-Step

Understanding the full lifecycle of a SLURM job — from submission to completion — is essential for debugging, optimization, and building robust training pipelines. Below is the complete sequence for a batch job (sbatch).



| sacct output ← query | ← |

3.2 Step 1: Job Submission

A user submits a job by running `sbatch` with a batch script. The script contains SLURM directives (`#SBATCH` lines) and the commands to execute. `sbatch` sends an RPC to `slurmctld` containing the full job specification.

```
#!/bin/bash
#SBATCH --job-name=llm-pretrain
#SBATCH --nodes=32
#SBATCH --ntasks-per-node=8          # 8 GPU processes per node
#SBATCH --gres=gpu:a100:8           # 8 A100s per node
#SBATCH --cpus-per-task=12          # 12 CPU cores per GPU process
#SBATCH --mem-per-cpu=8G
#SBATCH --time=48:00:00
#SBATCH --partition=gpu-large
#SBATCH --account=ml-team
#SBATCH --qos=high-priority
#SBATCH --output=logs/pretrain_%j.out
#SBATCH --error=logs/pretrain_%j.err
#SBATCH --mail-type=FAIL

module load cuda/12.2 nccl/2.19 pytorch/2.2

srun torchrun \
  --nnodes=$SLURM_NNODES \
  --nproc_per_node=$SLURM_NTASKS_PER_NODE \
  --rdzv_id=$SLURM_JOB_ID \
  --rdzv_backend=c10d \
  --rdzv_endpoint=$SLURM_NODELIST:29500 \
  train.py --config config/7B.yaml
```

3.3 Step 2: Validation & Queue Placement

`slurmctld` validates the submission against configured limits: wall-time maximums per partition, per-account resource limits, per-user job count limits, and QOS restrictions. If validation passes, the job is assigned a JobID and enters the PENDING state. `slurmctld` records the submission in `slurmdbd` asynchronously.

3.4 Step 3: Scheduling

The scheduling thread runs periodically and evaluates each pending job in priority order. For each job, the scheduler checks whether sufficient idle nodes satisfying the job's constraints (partition, feature flags, GRES, memory) are available. If yes, resources are allocated and the job transitions to RUNNING. If not, the job stays PENDING with a reason code (Resources, Priority, QOSMaxJobsPerUserLimit, etc.).

3.5 Step 4: Resource Allocation

When scheduling a job on N nodes, `slurmctld` marks those nodes as `ALLOCATED` in its state, reserving CPUs, memory, and GRES. The nodes are locked for the exclusive use of the job (or shared, depending on `OverSubscribe` policy). `slurmctld` constructs a launch message for each node.

3.6 Step 5: Job Launch

`slurmctld` sends `launch_tasks` RPCs to the `slurmd` on each assigned node simultaneously. Each `slurmd` forks a `slurmstepd` process that: creates cgroup hierarchies for CPU and memory isolation, sets GPU affinity, configures the network environment (MPI process topology, etc.), sets `SLURM_*` environment variables, and `exec`'s the job script or MPI launch command.

Key Environment Variables Injected

Variable	Value & Purpose
<code>SLURM_JOB_ID</code>	Unique job identifier — used for RNG seeds, checkpoint naming
<code>SLURM_NODELIST</code>	Comma-separated list of allocated node hostnames
<code>SLURM_NNODES</code>	Total number of allocated nodes
<code>SLURM_NTASKS</code>	Total number of MPI/distributed tasks
<code>SLURM_NTASKS_PER_NODE</code>	Tasks per node (GPU processes per node for DDP)
<code>SLURM_PROCID</code>	Global rank of this task (0 to <code>NTASKS-1</code>)
<code>SLURM_LOCALID</code>	Local rank within this node (0 to <code>NTASKS_PER_NODE-1</code>)
<code>SLURM_NODEID</code>	Index of this node in the allocation (0 to <code>NNODES-1</code>)
<code>CUDA_VISIBLE_DEVICES</code>	Set by GRES plugin to expose allocated GPUs to job

3.7 Job States

State	Code	Meaning
PENDING	PD	Waiting for resources; will show reason (e.g. Resources, Priority)
RUNNING	R	Actively executing on compute nodes
SUSPENDED	S	Temporarily paused (resources held, CPUs released)
COMPLETING	CG	Job finished; SLURM cleaning up processes/cgroups
COMPLETED	CD	Job finished normally with exit code 0
FAILED	F	Job exited with non-zero code or internal error
TIMEOUT	TO	Job exceeded its requested time limit
CANCELLED	CA	Cancelled by user (<code>scancel</code>) or admin
NODE_FAIL	NF	One or more assigned nodes failed during execution
PREEMPTED	PR	Preempted by a higher-priority job

State	Code	Meaning
OUT_OF_MEMORY	OOM	Job killed by cgroup OOM handler

Section 4 — Scheduling Algorithms

4.1 Priority Calculation

SLURM computes a numeric priority score for each pending job. The scheduler selects the highest-priority job that can be started. Priority is a weighted sum of several factors:

Priority = (w_age	× AgeFactor)	+
(w_fairshare	× FairShareFactor)	+
(w_jobsize	× JobSizeFactor)	+
(w_partition	× PartitionFactor)	+
(w_qos	× QOSFactor)	+
(w_assoc	× AssocFactor)	
# Each factor is normalized to [0.0, 1.0]		
# Weights configured in slurm.conf PriorityWeight* parameters		
# Typical production settings:		
PriorityWeightAge	= 1000	
PriorityWeightFairShare	= 10000	
PriorityWeightJobSize	= 0	
PriorityWeightPartition	= 0	
PriorityWeightQOS	= 2000	

AgeFactor

Increases linearly as a job waits in the queue. Prevents starvation: a low-priority job that has waited long enough will eventually jump to the front. The rate of increase is controlled by PriorityMaxAge (maximum age credit) and PriorityDecayHalfLife.

FairShareFactor

The most important factor in multi-user clusters. Based on the ratio of a group's allocated share to its actual historical usage. Accounts that have used less than their share get higher priority; heavy users are deprioritized. The fairshare algorithm is a tree-based calculation where each node's effective usage propagates up to parent accounts.

FairShare = 2 [^] (-effective_usage / raw_shares)
effective_usage decays over time (PriorityDecayHalfLife, default 7 days)
So a burst consumer today is deprioritized; after 7 days, half the
penalty fades; after 14 days, 75% fades.
Check current fairshare for an account:
sshare -a -u username
sshare -A ml-team --format=Account, NormShares, NormUsage, FairShare

4.2 Fair-Share Scheduling in Detail

SLURM's fair-share implementation uses a hierarchical account tree. At the leaf level, users have allocations under accounts; accounts may be nested under parent accounts. The raw shares at each level define the fractional entitlement. Usage rolls up from jobs through accounts.

```

Cluster (root account)
├─ ml-team      (500 shares, 60% of cluster)
│  └─ user-alice (200 shares within ml-team)
│     └─ user-bob (300 shares within ml-team)
└─ hpc-team     (300 shares, 40% of cluster)
   └─ user-carol (150 shares within hpc-team)
      └─ user-dave (150 shares within hpc-team)

# If ml-team has used 80% of its entitlement and hpc-team only 20%,
# hpc-team jobs get significantly higher priority regardless of
# individual submission times.

```

4.3 Backfill Scheduling

The main scheduler is conservative: it only starts jobs in strict priority order. This leads to poor utilization when the top job is large (needs 512 nodes) but only 64 nodes are available. Backfill scheduling resolves this by allowing lower-priority jobs to run ahead of higher-priority jobs, provided they complete before the higher-priority job can start.

```

Backfill algorithm (simplified):

1. Determine start time for each job in priority order
   (when will its required nodes become free?)

2. Build a node reservation map: for each node, track
   when it becomes free based on running job time limits

3. For each lower-priority pending job, in priority order:
   a. Can it start NOW on currently idle nodes?
   b. Will it FINISH before the reserved time of any
      higher-priority job on those nodes?
   c. If yes → launch immediately (backfill)

# Configure backfill:
SchedulerParameters=bf_resolution=60,bf_max_job_test=500,\
bf_window=24*60,bf_interval=30

```

Backfill Impact

On typical production clusters, backfill scheduling increases throughput by 15-40% compared to pure priority FIFO, by ensuring small jobs fill gaps while large jobs wait for sufficient nodes.

4.4 Preemption

SLURM supports job preemption: a higher-priority job can forcibly reclaim resources from lower-priority running jobs. The preemption mode is configured per-QOS pair.

Preemption Mode	Behavior	Use Case
CANCEL	Preempted job is immediately killed	When checkpoint is not available; scavenger queues

Preemption Mode	Behavior	Use Case
REQUEUE	Job is cancelled and resubmitted to queue	Fault-tolerant jobs that can restart from scratch
SUSPEND	Job CPUs released but memory/state held	Short preemptions, interactive sessions
CHECKPOINT	Job checkpointed then cancelled/requeued	Long jobs with DMTCP/CRIU checkpoint support

4.5 Reservations

SLURM allows admins to reserve nodes at a future time for specific users or accounts. Reservations block those nodes from regular scheduling during the reserved window. This is used for maintenance windows, priority access periods, and emergency reservations.

```
# Create a reservation for a major training run:
```

```
scontrol create reservation \  
  ReservationName=llm-sprint-2024 \  
  StartTime=2024-11-01T00:00:00 \  
  Duration=72:00:00 \  
  Nodes=gpu-[0001-0256] \  
  Users=alice,bob \  
  Flags=OVERLAP,IGNORE_JOBS
```

```
# View current reservations:
```

```
scontrol show reservation
```

```
# Submit to a reservation:
```

```
sbatch --reservation=llm-sprint-2024 train.sh
```

Section 5 — Resource Management

5.1 CPU Management

SLURM tracks CPUs at the socket, core, and thread (hyperthread) level. Jobs specify CPU requirements using `--cpus-per-task` (logical CPUs per process) and SLURM allocates them, setting CPU affinity via cgroups.

```
# CPU allocation example: 2 tasks, 16 CPUs each on a 2-socket server
sbatch --ntasks=2 --cpus-per-task=16 --mem-per-cpu=4G job.sh

# SLURM sets cpu affinity via taskset/cgroup:
# Task 0 → cores 0-15 (socket 0)
# Task 1 → cores 16-31 (socket 1)

# Check CPU binding:
srun --cpu-bind=verbose --ntasks=2 --cpus-per-task=16 numactl --show
```

5.2 GPU Management (GRES Framework)

GPUs and other accelerators are managed by SLURM's Generic Resource (GRES) framework. GRES allows arbitrary resource types to be defined, tracked, and allocated. GPUs are the most common GRES type in modern AI clusters.

GRES Configuration (gres.conf)

```
# /etc/slurm/gres.conf — on each compute node
# Automatically detect GPUs via NVML:
AutoDetect=nvml

# Or explicitly define GPUs:
Name=gpu Type=a100 File=/dev/nvidia0 Cores=0-23 Links=0,1,2,3,-1,-1,-1,-1
Name=gpu Type=a100 File=/dev/nvidia1 Cores=0-23 Links=1,0,2,3,-1,-1,-1,-1
Name=gpu Type=a100 File=/dev/nvidia2 Cores=0-23 Links=2,3,0,1,-1,-1,-1,-1
Name=gpu Type=a100 File=/dev/nvidia3 Cores=0-23 Links=3,2,1,0,-1,-1,-1,-1
Name=gpu Type=a100 File=/dev/nvidia4 Cores=24-47 Links=-1,-1,-1,-1,0,1,2,3
Name=gpu Type=a100 File=/dev/nvidia5 Cores=24-47 Links=-1,-1,-1,-1,1,0,2,3
Name=gpu Type=a100 File=/dev/nvidia6 Cores=24-47 Links=-1,-1,-1,-1,2,3,0,1
Name=gpu Type=a100 File=/dev/nvidia7 Cores=24-47 Links=-1,-1,-1,-1,3,2,1,0

# Links encodes NVLink connectivity: which GPUs are directly connected
# via NVLink vs PCIe. SLURM uses this for topology-aware GPU binding.

# MIG (Multi-Instance GPU) support:
Name=gpu Type=a100_mig3g File=/dev/nvidia0 AutoDetect=nvml
```

GPU Job Submission

```
# Request 4 A100 GPUs (any type)
sbatch --gres=gpu:4 job.sh

# Request 8 A100 GPUs specifically
```



```

sbatch --gres=gpu:a100:8 job.sh

# Full node with 8 GPUs, 96 CPUs, 320 GB RAM:
sbatch --nodes=1 --ntasks=8 --cpus-per-task=12 \
      --gres=gpu:a100:8 --mem=320G job.sh

# SLURM automatically sets CUDA_VISIBLE_DEVICES
# to expose only the allocated GPUs to the job.
# Example: CUDA_VISIBLE_DEVICES=2,3,6,7

```

5.3 Memory Management

SLURM enforces memory limits via Linux cgroups. When a job exceeds its memory limit, the kernel OOM killer terminates processes in the cgroup, and SLURM records the job as OUT_OF_MEMORY. Memory can be specified per node, per task, or per CPU.

```

--mem=320G          # Total memory per NODE
--mem-per-cpu=8G    # Memory per CPU core (total = cpus-per-task × 8G)
--mem-per-gpu=80G   # Memory per GPU (for GPU-memory-dominant workloads)

# Memory enforcement via cgroup:
# /sys/fs/cgroup/memory/slurm/uid_<UID>/job_<JOBID>/memory.limit_in_bytes

# Check memory usage of running jobs:
sstat --jobs=<jobid> --format=JobID,MaxRSS,MaxVMSize

```

5.4 NUMA Topology Awareness

Modern servers have multiple NUMA (Non-Uniform Memory Access) domains. Memory access to local NUMA node is 2-3× faster than to remote NUMA node. SLURM's task plugin (task/affinity or task/cgroup) can bind job processes to specific NUMA nodes, ensuring local memory access.

```

# Enable NUMA binding in slurm.conf:
TaskPlugin=task/cgroup
TaskPluginParam=Autobind=Cores

# Job: bind each MPI rank to a specific NUMA node:
srun --cpu-bind=socket --mem-bind=local --ntasks=2 ./mpi_app

# Verify NUMA topology of nodes:
numactl --hardware
lstopo --no-io # hwloc topology diagram

```

5.5 Heterogeneous Jobs

SLURM supports heterogeneous job allocations where different components of a job have different resource requirements. This is useful for training jobs that need a mix of GPU nodes and CPU-only preprocessing nodes.

```
# Heterogeneous job: 4 GPU nodes + 1 CPU preprocessing node
sbatch --nodes=4 --gres=gpu:8 --ntasks-per-node=8 : \
    --nodes=1 --ntasks=32 --mem=512G hetero_job.sh

# Inside the script, components are addressed by SLURM_HET_JOB_ID
# and SLURM_HET_JOB_OFFSET
```

Section 6 — SLURM Plugins & Extensibility

6.1 Plugin Architecture

SLURM is highly modular. Nearly every major subsystem is implemented as a dynamically loaded plugin (shared library, .so file). On startup, slurmd and slurmctld load the configured plugins. This allows vendors and HPC centers to customize behavior without modifying core SLURM code.

```
Plugin naming convention: <subsystem>/<implementation>
```

Examples:

```
SchedulerType = sched/backfill    # backfill scheduler
SelectType    = select/cons_tres  # consumable TRES resources
AuthType      = auth/munge        # Munge authentication
AccountingStorageType = accounting_storage/slurmdbd
TaskPlugin     = task/cgroup       # cgroup isolation
GresTypes      = gpu,fpga,nic      # GRES resource types
MpiType        = mpi/pmi2          # PMI2 for MPI
```

Plugin Category	Common Implementations	Purpose
Scheduler	sched/backfill, sched/builtin	Job scheduling algorithm
Select	select/cons_tres, select/linear	Resource selection & allocation
Authentication	auth/munge, auth/jwt	Daemon authentication
Accounting Storage	accounting_storage/slurmdbd, /none	Job accounting persistence
Job Submit	job_submit/lua	Custom job validation & modification on submit
Task	task/cgroup, task/affinity	CPU/memory isolation & binding
GRES	gres/gpu, gres/fpga, gres/nic	Generic resource tracking
MPI	mpi/pmi2, mpi/pmix, mpi/none	MPI process manager interface
Switch	switch/none, switch/infiniband	Network fabric setup
Topology	topology/tree, topology/3d_torus	Network topology awareness
Preempt	preempt/qos, preempt/partition_prio	Preemption policy
Priority	priority/multifactor	Job priority calculation

6.2 Job Submit Plugin (Lua)

The job_submit/lua plugin is one of the most powerful extensibility mechanisms. It allows site administrators to write Lua scripts that run on every job submission, enabling custom validation, default-setting, and policy enforcement without modifying SLURM source.

```
-- /etc/slurm/job_submit.lua
function slurm_job_submit(job_desc, part_list, submit_uid)
```

```
-- Enforce GPU jobs must specify --account
if job_desc.gres ~= nil and job_desc.account == nil then
    slurm.log_user("Error: GPU jobs must specify --account")
    return slurm.ERROR
end

-- Set default time limit if not specified
if job_desc.time_limit == slurm.NO_VAL then
    job_desc.time_limit = 60 -- 1 hour default
end

-- Enforce maximum wall time per partition
if job_desc.partition == "gpu-large" and job_desc.time_limit > 5760 then
    slurm.log_user("Max time on gpu-large is 4 days")
    return slurm.ERROR
end

return slurm.SUCCESS
end
```

6.3 PMIx / MPI Plugin

The MPI plugin determines how SLURM sets up the process management interface that MPI implementations use for process discovery and collective communication. PMIx (Process Management Interface for Exascale) is the modern standard, supported by Open MPI 4.x+, Intel MPI, and MPICH.

```
# Enable PMIx in slurm.conf
MpiType=mpi/pmix

# Submit an MPI job:
srun --mpi=pmix -n 256 ./mpi_application

# For PMI2 compatibility (older MPI):
srun --mpi=pmi2 -n 256 ./legacy_mpi_app
```

Section 7 — SLURM Configuration

7.1 slurm.conf — Main Configuration File

slurm.conf is the primary configuration file, read by all SLURM daemons on startup. It must be identical on every node in the cluster (typically distributed via configuration management: Ansible, Puppet, or a shared NFS mount).

```
# /etc/slurm/slurm.conf – Production GPU Cluster Example
```

```
# — Cluster Identity —————
```

```
ClusterName=gpu-cluster-prod
SlurmctldHost=ctrl01(10.0.0.1)
SlurmctldHost=ctrl02(10.0.0.2) # HA standby
```

```
# — Daemon Settings —————
```

```
SlurmUser=slurm
SlurmctldPort=6817
SlurmdPort=6818
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
SlurmctldDebug=info
SlurmdDebug=info
StateSaveLocation=/var/spool/slurm/ctld
SlurmdSpoolDir=/var/spool/slurm/d
```

```
# — Plugins —————
```

```
AuthType=auth/munge
SchedulerType=sched/backfill
SelectType=select/cons_tres
SelectTypeParameters=CR_Core_Memory
AccountingStorageType=accounting_storage/slurmdbd
AccountingStorageHost=db01
AccountingStoragePort=6819
TaskPlugin=task/cgroup,task/affinity
MpiType=mpi/pmix
ProctrackType=proctrack/cgroup
GresTypes=gpu,nic
```

```
# — Priority & Fair Share —————
```

```
PriorityType=priority/multifactor
PriorityWeightAge=1000
PriorityWeightFairShare=10000
PriorityWeightJobSize=0
PriorityWeightQOS=2000
PriorityMaxAge=7-0
PriorityDecayHalfLife=7-0
```

```
# — Scheduling —————
```

```
SchedulerParameters=bf_resolution=60,bf_max_job_test=1000,\
bf_window=2880,bf_interval=30,\
```

```

                                sched_max_job_start=200

# — Timeouts —————
SlurmctldTimeout=120
SlurmdTimeout=300
InactiveLimit=0
MinJobAge=300
KillWait=30
WaitTime=0

# — Topology —————
TopologyPlugin=topology/tree

# — Nodes —————
NodeName=gpu[0001-0256] \
  CPUs=96 Boards=1 SocketsPerBoard=2 CoresPerSocket=24 ThreadsPerCore=2 \
  RealMemory=983040 \
  Gres=gpu:a100:8 \
  State=UNKNOWN

NodeName=cpu[0001-0128] \
  CPUs=128 Boards=1 SocketsPerBoard=2 CoresPerSocket=32 ThreadsPerCore=2 \
  RealMemory=524288 \
  State=UNKNOWN

# — Partitions —————
PartitionName=gpu-large \
  Nodes=gpu[0001-0256] \
  Default=NO MaxTime=4-00:00:00 State=UP \
  OverSubscribe=NO \
  TRESBillingWeights="CPU=1,Mem=0.25G,GRES/gpu=100" \
  PriorityJobFactor=1

PartitionName=gpu-short \
  Nodes=gpu[0001-0128] \
  Default=YES MaxTime=04:00:00 State=UP \
  OverSubscribe=NO

PartitionName=cpu \
  Nodes=cpu[0001-0128] \
  Default=NO MaxTime=7-00:00:00 State=UP \
  OverSubscribe=FORCE:4

```

7.2 gres.conf — Generic Resource Configuration

gres.conf declares the specific GRES resources available on compute nodes. It can use auto-detection (NVML for GPUs) or explicit device declarations. The most critical configuration for GPU clusters.

```

# /etc/slurm/gres.conf – on GPU compute nodes

# Auto-detect all NVIDIA GPUs via NVML library:

```

```

AutoDetect=nvml

# This automatically creates entries equivalent to:
# Name=gpu Type=a100 File=/dev/nvidia0
# Name=gpu Type=a100 File=/dev/nvidia1
# ... for each detected GPU

# Also configure network adapters as GRES for IB-aware scheduling:
Name=nic Type=hdr File=/dev/infiniband/uverbs0 Count=1

# GPU frequency management:
# Use GPU defaults or set application clocks per job.

```

7.3 cgroup.conf — Resource Isolation

```

# /etc/slurm/cgroup.conf
CgroupPlugin=autodetect      # auto-detect cgroup v1 or v2
ConstrainCores=yes           # bind CPUs via cgroup.cpuset
ConstrainRAMSpace=yes        # enforce memory limits
ConstrainSwapSpace=yes       # prevent swap usage
ConstrainDevices=yes         # restrict GPU device access
AllowedRAMSpace=100          # % of requested mem before OOM
AllowedSwapSpace=0           # no swap allowed

```

7.4 topology.conf — Network Topology

```

# /etc/slurm/topology.conf – Fat-tree 3-tier switch topology
# Describes the switch hierarchy so SLURM can prefer topologically
# adjacent nodes for multi-node jobs (reduces inter-rack traffic).

# Leaf switches (connected to compute nodes)
SwitchName=leaf01 Nodes=gpu[0001-0032]
SwitchName=leaf02 Nodes=gpu[0033-0064]
SwitchName=leaf03 Nodes=gpu[0065-0096]
SwitchName=leaf04 Nodes=gpu[0097-0128]
SwitchName=leaf05 Nodes=gpu[0129-0160]
SwitchName=leaf06 Nodes=gpu[0161-0192]
SwitchName=leaf07 Nodes=gpu[0193-0224]
SwitchName=leaf08 Nodes=gpu[0225-0256]

# Spine switches (aggregate traffic from leaf switches)
SwitchName=spine01 Switches=leaf[01-04]
SwitchName=spine02 Switches=leaf[05-08]

# Core switch (connects spine switches)
SwitchName=core01 Switches=spine[01-02]

```

Section 8 — Networking & Cluster Topology

8.1 InfiniBand in HPC & AI Clusters

InfiniBand (IB) is the dominant high-performance interconnect in HPC and GPU training clusters. It provides microsecond latency, high bandwidth (200 Gb/s with HDR InfiniBand, 400 Gb/s with NDR), and RDMA (Remote Direct Memory Access) capabilities. RDMA allows one process to read/write the memory of another without involving the CPU, which is critical for efficient all-reduce operations in distributed training.

IB Generation	Speed	Typical Use
FDR (56 Gb/s)	56 Gb/s per port	Legacy clusters, pre-GPU scaling era
EDR (100 Gb/s)	100 Gb/s per port	Common in late 2010s HPC clusters
HDR (200 Gb/s)	200 Gb/s per port	Current standard for GPU training clusters
NDR (400 Gb/s)	400 Gb/s per port	Emerging; NVLink + NDR for next-gen LLMs
XDR (800 Gb/s)	800 Gb/s per port	Future specification (2024+)

8.2 SLURM Topology Awareness

When topology/tree plugin is enabled and topology.conf is configured, SLURM's resource selection algorithm prefers nodes within the same switch subtree to minimize the number of network hops between job processes. For a 256-node job on a 2-level fat-tree cluster, SLURM will try to allocate all nodes within the same spine group before spreading across spines, reducing congestion and improving all-reduce bandwidth.

```
# Topology selection strategies:
# TopologyParam=SwitchAsNodeRank    - rank nodes by switch proximity
# TopologyParam=TopoOptional        - relax topology constraints if
#                                   needed to start the job sooner

# For a 4-node job on a fat-tree cluster:
# SLURM prefers: all 4 nodes on same leaf switch
# (0 hop between nodes, only local switch bandwidth used)
# vs. 4 nodes on different leaves:
# (traffic must traverse 2 spine hops - lower effective bandwidth)
```

8.3 RDMA & NCCL Integration

NCCL (NVIDIA Collective Communications Library) uses RDMA over InfiniBand (RoCE or native IB) for all-reduce and other collective operations. SLURM itself does not configure RDMA — this is handled by the network stack (OFED drivers, rdma-core) — but proper GRES configuration ensures GPUs and InfiniBand ports are allocated as a unit, placing GPU processes on nodes with attached IB HCAs.

```
# Ensure NCCL uses InfiniBand for collective operations:
export NCCL_DEBUG=INFO
export NCCL_IB_DISABLE=0
```



```
export NCCL_IB_HCA=mlx5_0,mlx5_1 # IB HCA device names
export NCCL_SOCKET_IFNAME=ib0    # IP interface for NCCL control plane
export NCCL_TREE_THRESHOLD=0     # Use ring algorithm for all sizes

# Alternatively, use UCX for MPI collective operations:
export UCX_TLS=rc,ud,sm,self
export UCX_IB_GPU_DIRECT_RDMA=1  # GPU-Direct RDMA for zero-copy
```

8.4 High-Speed Ethernet (RoCE)

RDMA over Converged Ethernet (RoCE) brings RDMA capabilities to 100/200/400 GbE Ethernet infrastructure. RoCE v2 is widely deployed as a lower-cost alternative to native InfiniBand, used heavily in cloud provider GPU clusters (AWS EFA, GCP GPUDirect, Azure RDMA). SLURM cluster configuration for RoCE is similar to IB — the difference lies in the network driver level.

Section 9 — Monitoring, Logging & Accounting

9.1 slurmdbd & Accounting Database

slurmdbd is the SLURM database daemon. It provides a durable store for all job accounting records, enabling retrospective queries, usage reporting, fair-share calculations, and limit enforcement. The underlying database is MySQL/MariaDB.

```
# Key tables in the slurm_acct_db database:
# cluster_event_table    - node up/down events
# job_table              - one row per job: start, end, CPUs, GPUs used
# step_table             - per job-step resource usage
# assoc_table            - user/account/partition associations
# usage_hour_table       - hourly rollup of resource consumption
# qos_table              - Quality of Service definitions

# Example sacct queries:
# All jobs in last 7 days for ml-team account:
sacct -A ml-team --starttime=$(date -d "7 days ago" +%Y-%m-%d) \
      --format=JobID,User,Account,Start,End,Elapsed,CPUTimeRAW,\
      NCPUS,AllocTRES,ExitCode,State

# GPU utilization per job:
sacct --jobs=123456 --format=JobID,AllocGRES,ReqGRES,TRESUsageInAve
```

9.2 Job Efficiency Metrics

A critical cluster health metric is job efficiency: the ratio of CPU/GPU time actually used to what was allocated. Inefficient jobs waste cluster resources and reduce fairness. seff (SLURM efficiency tool) provides a quick summary.

```
$ seff 123456
Job ID: 123456
Cluster: gpu-cluster-prod
User/Group: alice/ml-team
State: COMPLETED (exit code 0)
Nodes: 32
Cores per node: 96
CPU Utilized: 02-14:22:45
CPU Efficiency: 87.34% of 02-22:24:00 core-walltime
Job Wall-clock time: 18:03:00
Memory Utilized: 9.43 TB
Memory Efficiency: 92.1% of 10.00 TB

# GPU efficiency requires additional tooling:
# nvidia-smi dmon, DCGM (Data Center GPU Manager)
```

9.3 Logging Architecture

SLURM logging is configured per daemon. Logs are written to files on the controller and compute nodes. For production clusters, logs are shipped to a centralized logging system (ELK Stack, Splunk, or Loki) for aggregation, alerting, and audit trails.

Log Source	Default Location	Key Events
slurmctld.log	/var/log/slurm/slurmctld.log	Job submit/start/end, scheduling decisions, node state changes
slurmd.log	/var/log/slurm/slurmd.log (each node)	Job step launch, resource allocation, node health
slurmdbd.log	/var/log/slurm/slurmdbd.log	Database writes, connection errors, accounting records
audit.log	Configurable	Admin commands (scontrol), privilege escalations

9.4 External Monitoring Integration

For production clusters, slurmctld metrics are scraped by monitoring systems. The Prometheus slurm_exporter (open-source) exposes cluster metrics in Prometheus format, enabling Grafana dashboards.

```
# Prometheus slurm_exporter key metrics:
slurm_nodes_alloc      - number of allocated nodes
slurm_nodes_idle       - number of idle nodes
slurm_nodes_down       - number of down nodes
slurm_jobs_running     - number of running jobs
slurm_jobs_pending     - number of pending jobs
slurm_partition_jobs_pending - pending jobs per partition
slurm_cpus_total       - total CPUs in cluster
slurm_cpus_alloc       - allocated CPUs

# GPU utilization (via DCGM exporter, not SLURM native):
DCGM_FI_DEV_GPU_UTIL   - GPU SM utilization %
DCGM_FI_DEV_MEM_COPY_UTIL - GPU memory bandwidth utilization %
DCGM_FI_DEV_POWER_USAGE - GPU power draw (Watts)
DCGM_FI_DEV_NVLINK_BANDWIDTH_TOTAL - NVLink bandwidth
```

9.5 Failure Recovery

SLURM is designed to survive controller restarts without losing jobs. The recovery procedure relies on the state files written by slurmctld and the accounting records in slurmdbd.

```
# Recovery sequence on controller restart:
# 1. slurmctld reads StateSaveLocation checkpoint files
# 2. Reconstructs in-memory job queue and node states
# 3. Reconnects to slurmd daemons on compute nodes
# 4. Running jobs continue; no interruption to compute nodes
# 5. Pending jobs resume scheduling
```

```
# Controller HA failover:  
# Primary slurmctld fails → standby detects loss within SlurmctldTimeout  
# Standby promotes itself → clients reconnect automatically  
# Failover time: typically 30-120 seconds  
  
# Force a manual failover for maintenance:  
scontrol takeover    # On the standby controller
```

Section 10 — SLURM for AI/ML Training

10.1 GPU Cluster Workflow

The canonical AI/ML training workflow on a SLURM cluster involves submitting a batch job that uses `srun` (or `torchrun`) to launch distributed training processes, with each process pinned to a specific GPU via SLURM's GRES allocation and `CUDA_VISIBLE_DEVICES`.

```
#!/bin/bash
#SBATCH --job-name=gpt4-finetune
#SBATCH --nodes=16
#SBATCH --ntasks-per-node=8
#SBATCH --gres=gpu:a100:8
#SBATCH --cpus-per-task=12
#SBATCH --mem-per-cpu=8G
#SBATCH --time=12:00:00
#SBATCH --partition=gpu-large
#SBATCH --exclusive # dedicated nodes, no sharing

# Environment setup
source /opt/conda/etc/profile.d/conda.sh
conda activate llm-env

# Distributed training with torchrun + SLURM
export MASTER_ADDR=$(scontrol show hostnames $SLURM_JOB_NODELIST | head -n 1)
export MASTER_PORT=29500
export WORLD_SIZE=$((SLURM_NNODES * SLURM_NTASKS_PER_NODE))

srun torchrun \
  --nnodes=$SLURM_NNODES \
  --nproc_per_node=$SLURM_NTASKS_PER_NODE \
  --rdzv_id=$SLURM_JOB_ID \
  --rdzv_backend=c10d \
  --rdzv_endpoint=$MASTER_ADDR:$MASTER_PORT \
  --max_restarts=3 \
  train_llm.py \
  --model-config config/13B.yaml \
  --data-path /data/pretraining \
  --checkpoint-dir /checkpoints/$SLURM_JOB_ID \
  --resume-from-checkpoint latest
```

10.2 PyTorch Distributed Data Parallel (DDP)

PyTorch DDP is the standard approach for single-machine-multi-GPU and multi-machine-multi-GPU training. Each GPU runs an identical copy of the model; gradients are synchronized via all-reduce after each backward pass. SLURM provides the infrastructure for process launch and GPU allocation.

```
# Process mapping (16 nodes × 8 GPUs = 128 ranks):
# SLURM_PROCID = global rank (0-127)
# SLURM_LOCALID = local rank within node (0-7)
```

```
# SLURM_NODEID = node index (0-15)

# In PyTorch code:
import os
import torch.distributed as dist

dist.init_process_group(
    backend="nccl",
    init_method="env://",
    world_size=int(os.environ["WORLD_SIZE"]),
    rank=int(os.environ["SLURM_PROCID"]),
)
torch.cuda.set_device(int(os.environ["SLURM_LOCALID"]))

model = MyModel().cuda()
model = torch.nn.parallel.DistributedDataParallel(model)
```

10.3 Megatron-LM & Tensor Parallelism on SLURM

Very large models (70B+ parameters) require tensor parallelism and pipeline parallelism in addition to data parallelism. Megatron-LM is the standard framework for this. The job configuration must account for the interplay between tensor-parallel groups, pipeline-parallel groups, and data-parallel groups.

```
# Megatron-LM 3D parallelism on SLURM:
# Tensor Parallel (TP) = 8 → 8 GPUs share one transformer layer
# Pipeline Parallel (PP) = 4 → 4 pipeline stages
# Data Parallel (DP) = 8 → 8 copies of model
# Total GPUs = TP × PP × DP = 8 × 4 × 8 = 256 GPUs = 32 nodes

srun python pretrain_gpt.py \
    --tensor-model-parallel-size 8 \
    --pipeline-model-parallel-size 4 \
    --num-layers 96 \
    --hidden-size 12288 \
    --num-attention-heads 96 \
    --seq-length 4096 \
    --max-position-embeddings 4096 \
    --micro-batch-size 2 \
    --global-batch-size 2048 \
    --train-iters 500000
```

10.4 Horovod Integration

Horovod is Uber's distributed training framework that wraps TensorFlow, PyTorch, and MXNet with an efficient all-reduce implementation. Horovod uses MPI under the hood, which integrates cleanly with SLURM's MPI process management.

```
#!/bin/bash
#SBATCH --nodes=8
#SBATCH --ntasks-per-node=8
```

```
#SBATCH --gres=gpu:8

# Horovod with OpenMPI via SLURM:
srun --mpi=pmix python train_horovod.py

# In training code:
import horovod.torch as hvd
hvd.init()
torch.cuda.set_device(hvd.local_rank())
optimizer = hvd.DistributedOptimizer(optimizer)
hvd.broadcast_parameters(model.state_dict(), root_rank=0)
```

10.5 Fault Tolerance & Checkpointing

Long training runs (multi-day) are vulnerable to hardware faults, preemption, and software crashes. Robust training pipelines implement periodic checkpointing and auto-resume.

```
# Auto-requeue on preemption or node failure:
#SBATCH --requeue
#SBATCH --open-mode=append      # Append to log on restart

# Detect restarts in training code:
RESTART_COUNT=${SLURM_RESTART_COUNT:-0}

if [ $RESTART_COUNT -gt 0 ]; then
    CHECKPOINT_ARG="--resume-from-checkpoint /checkpoints/latest"
else
    CHECKPOINT_ARG=""
fi

srun python train.py $CHECKPOINT_ARG

# Best practice: checkpoint every N steps (e.g. every 30 min)
# Save to parallel filesystem: /checkpoints/$SLURM_JOB_ID/step_N/
```

Section 11 — How Cloud Providers Use SLURM

11.1 Google Vertex AI Training Clusters

Google Vertex AI Training Clusters (introduced in 2024) offer managed HPC-like clusters for large-scale ML training. They support SLURM as the workload manager, running on Google's high-performance TPU and GPU infrastructure connected by their Jupiter network fabric.

In Vertex AI training clusters, SLURM runs in a GCP-native configuration: the controller runs as a managed GCE VM, compute nodes are provisioned on-demand as GCE VMs (or can be pre-provisioned reservations), and storage is provided by Filestore (NFS) or Cloud Storage (FUSE-mounted). Google manages the SLURM installation, controller high-availability, and node lifecycle.

```
# Vertex AI Training Cluster SLURM job submission:
# Submit from within the cluster (via SSH to login node)
# or via Vertex AI SDK:

from google.cloud import aiplatform
aiplatform.init(project="my-project", location="us-central1")

job = aiplatform.CustomJob(
    display_name="llm-training",
    worker_pool_specs=[{
        "machine_spec": {
            "machine_type": "a2-highgpu-8g",
            "accelerator_type": "NVIDIA_TESLA_A100",
            "accelerator_count": 8,
        },
        "replica_count": 32,
        "container_spec": {"image_uri": "gcr.io/my-project/trainer:latest"},
    }],
)
job.submit()

# Or via standard sbatch on the cluster login node:
sbatch --nodes=32 --gres=gpu:8 --partition=gpu train.sh
```

11.2 AWS ParallelCluster

AWS ParallelCluster is Amazon's managed HPC cluster solution. It provisions EC2 instances (Hpc6a, P4d, P4de, P5 instances with A100/H100 GPUs) connected by Elastic Fabric Adapter (EFA) — AWS's RDMA-capable networking. SLURM is the default scheduler. Compute nodes auto-scale: when jobs are submitted, ParallelCluster provisions new EC2 instances; when idle, they terminate to reduce cost.

```
# AWS ParallelCluster config (pcluster.yaml):
Scheduling:
  Scheduler: slurm
  SlurmQueues:
    - Name: gpu-queue
```



```

Networking:
  SubnetIds: [subnet-xxxxx]
  PlacementGroup:
    Enabled: true          # cluster placement for low latency
  ComputeResources:
    - Name: p4d-nodes
      Instances:
        - InstanceType: p4d.24xlarge # 8× A100, EFA, 400 Gb/s
      MinCount: 0
      MaxCount: 64
      Efa:
        Enabled: true
        GdrSupport: true    # GPU-Direct RDMA

```

11.3 Azure CycleCloud

Azure CycleCloud orchestrates SLURM (or other schedulers) on Azure VM scale sets. It supports HC/HB series VMs for HPC (InfiniBand via SR-IOV) and NC/ND series for GPU workloads. CycleCloud provides autoscaling, cost management, and integration with Azure Active Directory for user authentication.

11.4 SLURM vs. Kubernetes for ML Workloads

The choice between SLURM and Kubernetes for ML training infrastructure involves fundamental trade-offs. Many organizations deploy both and route workloads accordingly.

Criterion	SLURM	Kubernetes
Tightly-coupled MPI training	Excellent — native gang scheduling	Requires MPI operator, complex
GPU topology awareness	Excellent — NVLink topology in gres.conf	Limited without custom device plugins
InfiniBand / RDMA	Native bare-metal access	SR-IOV device plugin (complex setup)
Container support	Via Singularity/Apptainer, Podman	Native Docker/OCI containers
Elastic scaling	Manual or AWS ParallelCluster autoscale	Native horizontal pod autoscaling
Multi-tenancy	User accounts, fair-share, QOS	Namespaces, RBAC, resource quotas
Job preemption	First-class, GPU-aware	Priority preemption, less granular
Serving / inference	Not designed for this	Excellent — native service routing
Developer familiarity	HPC community well-versed	Broad industry adoption
Operator overhead	Lower for HPC workloads	Complex for HPC, simpler for services

Section 12 — Scaling SLURM

12.1 Large-Scale Deployment Considerations

SLURM has been deployed at scale exceeding 700,000 cores at facilities like Lawrence Livermore National Laboratory (Sierra, Lassen supercomputers). At this scale, the architecture must be carefully tuned to avoid controller bottlenecks.

Controller Performance Tuning

```
# slurm.conf tuning for large clusters (10k+ nodes):

# Increase scheduler loop speed:
SchedulerParameters=bf_resolution=30,bf_max_job_test=2000,\
    sched_min_interval=2000000

# Increase slurmctld thread counts:
SlurmctldParameters=max_switch_wait=300,idle_on_node_suspend

# Reduce heartbeat load from compute nodes:
SlurmdTimeout=300      # tolerate 5min of no heartbeat before DOWN

# Limit DB query batch sizes:
AccountingStorageParameters=job_rollup=yes

# Use node health check plugin to filter bad nodes automatically:
HealthCheckProgram=/etc/slurm/nhc.sh
HealthCheckInterval=300
```

12.2 Controller High Availability

SLURM supports active/standby controller pairs for high availability. Only one slurmctld is active at a time. The standby monitors the primary via a heartbeat and promotes itself if the primary becomes unreachable.

```
# slurm.conf: HA controller configuration
SlurmctldHost=ctrl01(10.0.0.1) # Primary
SlurmctldHost=ctrl02(10.0.0.2) # Standby

# Both controllers must share:
# - StateSaveLocation (via shared NFS or DRBD replicated volume)
# - munge.key (same munge key on both)
# - slurm.conf (identical)

# Failover time: ~SlurmctldTimeout seconds (default 120s)
# Running jobs on compute nodes are NOT interrupted during failover

# Manual failover:
scontrol takeover # Run on standby to force promotion
```

12.3 SLURM Federation

SLURM federation allows multiple independent SLURM clusters to share a unified job namespace. Jobs can be submitted to a federated cluster and automatically routed to the cluster with available resources. This enables cross-datacenter or cross-facility scheduling.

```
# Create a federation linking two clusters:
sacctmgr add federation fed01 clusters=cluster-east,cluster-west

# Submit a federated job (will run on whichever cluster has resources):
sbatch --clusters=all --nodes=64 --gres=gpu:8 train.sh

# View federated job status:
squeue --federation

# Each cluster maintains its own slurmctld and slurmdbd;
# federation adds a meta-scheduler layer that routes jobs.
```

12.4 Multi-Cluster Scheduling

```
# Direct job submission to a specific remote cluster:
sbatch --clusters=cluster-west --nodes=128 train.sh

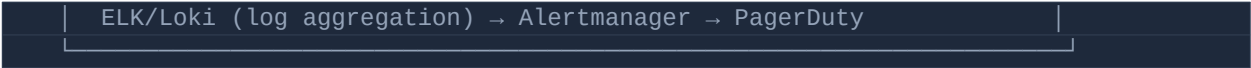
# Check queue on multiple clusters simultaneously:
squeue --clusters=cluster-east,cluster-west --user=alice

# sacct across clusters:
sacct --clusters=all --starttime=2024-01-01 --user=alice
```

Section 13 — Example Production Architecture

13.1 Reference Architecture: 256-Node A100 GPU Training Cluster





13.2 Cluster Specifications

Component	Specification	Count
GPU Compute Nodes	DGX A100: 8× A100 SXM4 80GB, 2× AMD EPYC 7742 (96 cores), 1 TB DDR4, 1× HDR IB	256 nodes
Total GPUs	NVIDIA A100 SXM4 80 GB	2,048 GPUs
Total CPU Cores	96 cores per node × 256 nodes	24,576 cores
Total GPU Memory	80 GB per GPU × 2048	163 TB HBM2e
IB Network	HDR 200 Gb/s per node, Fat-Tree non-blocking	256 ports
Storage	Lustre parallel FS, 16× OSS	2 PB / 400 GB/s
Controller	HA pair, 2× 64-core EPYC, 512 GB	2 VMs
Login Nodes	2× 48-core, 256 GB, 10 GbE	2 nodes

Section 14 — Comparison with Kubernetes, Ray & YARN

14.1 Comprehensive Scheduler Comparison

Feature	SLURM	Kubernetes	Ray	YARN (Hadoop)
Primary use case	HPC batch jobs, GPU training	Containerized microservices	Distributed Python, RL, tuning	Big data (Spark, Flink)
Scheduling model	Batch priority + backfill	Gang with priority classes	Task graph + actor model	Capacity-based FIFO
Resource unit	CPU/GPU/memory per node	Pod (CPU+mem requests)	Task (CPU fraction + GPU)	Container (vCores + MB)
GPU support	Excellent, topology-aware	Good via device plugin	Good via GPU tasks	Limited
Fault tolerance	Job requeue, HA controller	Pod restart policy	Actor retry, placement retry	Task retry in AM
Container native	Via Singularity/Podman	Yes (Docker/OCI)	Python-native, containers optional	Yes (YARN cgroups)
Multi-tenancy	Accounts, QOS, fair-share	Namespaces + RBAC	Project isolation	Queues + ACLs
Autoscaling	Via ParallelCluster/CycleCloud	Native HPA/VPA/Cluster-AS	Autoscaler built-in	YARN capacity auto-expand
MPI support	Native (PMIx, PMI2)	MPI operator (kubeflow)	Fragile, workarounds	Limited
Latency	Low (<1s job start)	Higher (pod scheduling)	Low (in-process)	Higher

14.2 Decision Framework

Choosing the Right Scheduler

Use SLURM when: jobs are tightly-coupled MPI or GPU training workloads, InfiniBand fabric is present, bare-metal performance isolation is required, or you have a dedicated HPC cluster. Use Kubernetes when: workloads are containerized, you need microservice-style deployment and scaling, the team is cloud-native, or you are running inference serving alongside training. Use Ray when: the workload is Python-native, involves hyperparameter tuning (Ray Tune), reinforcement learning, or complex distributed compute graphs.

14.3 Hybrid Architectures

Production AI infrastructure at scale often uses both SLURM and Kubernetes. A common pattern is:

- SLURM cluster: dedicated GPU training cluster for large-scale pretraining and fine-tuning
- Kubernetes cluster: model registry, training experiment tracking (MLflow), and inference serving
- Shared storage: Lustre or NFS accessible from both clusters for model artifacts and datasets
- CI/CD pipeline: Kubernetes-native CI triggers SLURM job submissions via REST API or SSH

Section 15 — End-to-End Distributed Training Workflow

15.1 Complete Workflow: LLM Pre-Training on SLURM

This section walks through the complete lifecycle of a production large language model pre-training run on a SLURM GPU cluster, from data preparation through to trained model artifacts.

== PHASE 0: Infrastructure Preparation ==

1. Admin configures cluster:
 - slurm.conf: nodes, partitions, QOS, fair-share accounts
 - gres.conf: GPU auto-detect via NVML
 - topology.conf: leaf/spine switch hierarchy
 - cgroup.conf: memory + device isolation
2. Admin provisions storage:
 - Lustre filesystem mounted at /data and /checkpoints
 - Dataset pre-staged: /data/c4-en/ (1.5 TB compressed)
3. User creates SLURM account:


```
sacctmgr add user alice account=ml-team
sacctmgr modify account ml-team set GrpTRES=gres/gpu=512
```

== PHASE 1: Environment & Code Setup ==

```
# User sets up container / environment:
module load singularity/3.11
singularity pull nvcv.io/nvidia/pytorch:24.01-py3

# Or uses conda environment on shared filesystem:
conda create -n llm python=3.11
pip install torch==2.2 transformers megatron-lm
```

== PHASE 2: Job Submission ==

```
sbatch pretrain_13B.sh
# Output: Submitted batch job 789012
```

== PHASE 3: Queue & Scheduling ==

```
# Check job status:
squeue --job=789012 --format="%10i %10u %12P %12j %8T %12M %12l %6D %R"
#      JOBID      USER  PARTITION  NAME      STATE      TIME  TIME_LIMI
NODES REASON
# 789012      alice  gpu-large  llm-pre  PENDING      0:00  48:00:00    32
Resources

# Fair-share check:
sshare -u alice
# Shows FairShare=0.87 (alice has used 13% of her allocation - high priority)
```



```
# Backfill scheduler finds 32 nodes available:
# Job transitions PENDING → RUNNING at T+12min

===== PHASE 4: Execution =====

# SLURM actions on all 32 nodes simultaneously:
# 1. slurmctld → launch_tasks RPC → each slurmd
# 2. slurmd forks slurmstepd per node
# 3. slurmstepd sets:
#   - CUDA_VISIBLE_DEVICES=0,1,2,3,4,5,6,7
#   - SLURM_PROCID, SLURM_LOCALID, SLURM_NODEID
#   - cgroup: CPUs 0-95, GPUs /dev/nvidia[0-7]
# 4. slurmstepd exec: torchrun with rdvz on node 0:29500

# Training begins: 256 GPU processes, TP=8, PP=4, DP=8
# NCCL all-reduce over IB HDR fabric
# Checkpoint every 500 steps → /checkpoints/789012/step_N.pt

===== PHASE 5: Monitoring During Training =====

# Real-time GPU utilization:
srun --jobid=789012 --pty bash
nvidia-smi dmon -s pucvmet -d 5

# Job step resource usage:
sstat --jobs=789012 --format=JobID,MaxRSS,MaxVMSize,MaxDiskRead

# Monitor NCCL communication (check IB bandwidth):
srun --jobid=789012 ibstat | grep Active

===== PHASE 6: Completion =====

# Job finishes after 47 hours (under 48h limit)
# Exit code 0 → State: COMPLETED

# Accounting record written to slurmdbd:
sacct --jobs=789012 \
      --format=JobID,Account,AllocNodes,AllocTRES,Elapsed,CPUTimeRAW,\
              TRESUsageInAve,ExitCode,State

# CPU/GPU efficiency check:
seff 789012
# CPU Efficiency: 91.2% GPU Efficiency: 87.4%

===== PHASE 7: Model Artifacts =====

# Final checkpoint at /checkpoints/789012/final/
# Convert to HuggingFace format:
python convert_megatron_to_hf.py \
  --input /checkpoints/789012/final \
  --output /models/gpt-13B-v1
```

```
# Push to model registry (MLflow / W&B):
mlflow models register --model-uri /models/gpt-13B-v1 \
  --name gpt-13B --version 1
```

15.2 Troubleshooting Common Issues

Issue	Diagnosis Command	Likely Cause & Fix
Job stuck PENDING forever	<code>squeue -j JOB --format="%R"</code>	Reason=Resources: not enough GPUs free. Reason=Priority: lower priority than queued jobs. Check <code>sinfo -p</code> partition.
Job fails immediately (exit code 1)	<code>cat logs/job_JOBID.err</code>	Module not loaded, Python path wrong, or NCCL init failure. Check <code>MASTER_ADDR</code> is reachable.
OOM killed	<code>sacct -j JOB --format=State,ExitCode</code>	State=OUT_OF_MEMORY. Increase <code>--mem</code> or <code>--mem-per-gpu</code> . Check gradient checkpointing.
NCCL timeout	<code>cat logs/job_JOBID.err grep NCCL</code>	IB fabric issue or firewall blocking port 29500. Check <code>NCCL_DEBUG=INFO</code> output.
Node goes DOWN during job	<code>scontrol show node NODE</code>	Hardware failure. Job may requeue if <code>--requeue</code> set. Check <code>slurmd.log</code> on failed node.
Slow training throughput	<code>nvidia-smi dmon</code> during job	GPU utilization <80%: data loader bottleneck or CPU-GPU sync overhead. Check Lustre throughput.

15.3 Key SLURM Commands Quick Reference

— SUBMISSION —	
<code>sbatch script.sh</code>	# Submit batch job
<code>srunc --pty --gres=gpu:1 bash</code>	# Interactive GPU shell
<code>salloc -N 4 --gres=gpu:8</code>	# Allocate 4 nodes interactively
— MONITORING —	
<code>squeue -u \$USER</code>	# My jobs
<code>squeue -p gpu-large</code>	# All jobs in partition
<code>sinfo -p gpu-large -o "%N %C %m"</code>	# Nodes: CPUs and memory
<code>scontrol show job JOBID</code>	# Full job details
<code>scontrol show node NODE</code>	# Node details (GPUs, state)
<code>sstat -j JOBID --format=MaxRSS</code>	# Live memory stats
— ACCOUNTING —	
<code>sacct -u \$USER -S 2024-01-01</code>	# My jobs since Jan 1
<code>sshare -u \$USER</code>	# My fair-share score
<code>sreport cluster utilization</code>	# Cluster-wide utilization report
— ADMIN —	
<code>scontrol update NodeName=gpu0042 State=DRAIN Reason="disk failure"</code>	

```
scontrol update NodeName=gpu0042 State=RESUME
scontrol hold JOBID                # Prevent job from starting
scontrol release JOBID             # Allow held job to start
scontrol requeue JOBID             # Requeue a running job
scancel --user=alice               # Cancel all of alice's jobs
```

Document Summary

This document has covered SLURM from foundational scheduling theory through production deployment architecture for GPU training clusters. Key takeaways: (1) SLURM excels at tightly-coupled HPC and GPU training workloads where Kubernetes struggles. (2) The fair-share + backfill combination is the workhorse of production cluster efficiency. (3) GPU topology configuration via `gres.conf` and `topology.conf` is critical for AI/ML performance. (4) Cloud providers wrap SLURM in managed infrastructure (ParallelCluster, CycleCloud, Vertex AI) but the core scheduling model remains identical. (5) Production deployments require HA controllers, slurmdbd accounting, and external monitoring integration.

END OF DOCUMENT — SLURM Infrastructure Reference v1.0